Documento de Engenharia de Software

Feature Request & Product Roadmap Management System

Diagrama de Sequência 2 — Aprovar Pedido e Gerar User Story

Ano Letivo 2026/27

1 Objetivo

Este diagrama mostra como um pedido triado é aprovado e transformado numa User Story com ajuda da IA. Cobre a avaliação de esforço e impacto (US 1.6) e a conversão automática em User Story com um clique (US 1.3).

2 Participantes

- **Gestor de Produto** (persona Tiago): aprova o pedido e revê a User Story.
- **Aplicação Web (React)**, **API Back-end (Django)** e **Base de Dados (PostgreSQL)**: os contentores do sistema (C4 nível 2).
- **Serviço de IA**: gera o esboço da User Story e dos critérios de aceitação.

3 Descrição do fluxo

3.1 Aprovação (passos 1 a 10)

O Gestor de Produto define o esforço e o impacto do pedido e clica em "Aprovar". A API obtém o pedido e verifica a avaliação. Se faltar o esforço ou o impacto, responde `400 ''Avaliação obrigatória''`. Caso contrário, executa `pedido.aprovar()`, guarda o estado `APROVADO` e a aplicação mostra o botão "Criar User Story".

3.2 Geração da User Story (passos 11 a 22)

Ao clicar em "Criar User Story", a API chama `pedido.gerarUserStory()`, que pede ao Serviço de IA um esboço no formato "Como..., eu preciso de..., para que..." com critérios Dado/-Quando/Então. O pedido é repetido no máximo duas vezes (fragmento `loop`) até a resposta ter o formato correto. Se a IA falhar nas duas tentativas, a story é criada com a descrição original do pedido e sem critérios. Em qualquer caso, a User Story é guardada no estado `A_FAZER`, no último lugar do Product Backlog (`ordemBacklog`), ligada ao pedido de origem e aos seus anexos.

3.3 Revisão (passos 23 a 31)

O Gestor de Produto revê o esboço. Pode editá-lo e confirmá-lo, ficando a story no Product Backlog, onde pode depois ser reordenada, ou descartá-lo, sendo a story removida.

4 Decisões e regras

- **Avaliação obrigatória antes de aprovar:** garante que todas as decisões de aprovação são comparáveis na matriz de Esforço vs. Impacto (US 1.6).
- **Limite de duas tentativas à IA:** evita esperas longas; o utilizador recebe sempre uma story, nunca um erro sem saída.
- **Alternativa sem IA:** usar a descrição original cumpre o critério da US 1.3 ("mapeando a descrição original e os respetivos anexos") mesmo quando a IA está indisponível.
- **Ligação ao pedido de origem:** é o que permite ao Programador abrir a "Origem do Pedido" (US 3.1).
- **Story sem critérios é válida:** uma User Story pode existir no Product Backlog sem critérios (multiplicidade `0..*` no diagrama de classes); só não pode entrar num Sprint enquanto não tiver pelo menos um, regra verificada por `estaPronta()` (Diagrama 4).

- **Sem estados de rascunho:** a story nasce em `A_FAZER` e fica no Product Backlog; só entra num Sprint depois de cumprir a Definition of Ready (Diagrama 4).

5 Ligação aos outros artefactos

- **Casos de uso (EDS-12):** "Avaliar esforço e impacto", "Aprovar pedido" e "Gerar User Story" (Gestor de Produto).
- **Classes (EDS-13):** Pedido (`aprovar()`, `gerarUserStory()`), UserStory, CriterioAceitacao e Anexo.
- **Diagramas de estados:** Pedido Em Triagem → Aprovado; User Story criada em A Fazer.

O diagrama está na página seguinte, em formato vetorial, e podem ser ampliados sem perder qualidade. O código PlantUML que os gera está nas últimas páginas e pode ser colado em `planttext.com` para o editar.

Diagrama de Sequência 2 — Aprovar pedido e gerar User Story (US 1.6, US 1.3)

Gestor de Produto
Aplicação Web (React)
API Back-end (Django)
Base de Dados (PostgreSQL)
Serviço de IA

Aprovação

1 define esforço e impacto e clica "Aprovar"
2 POST /pedidos/{id}/aprovar
3 obter Pedido
4 pedido

alt [esforço ou impacto em falta]
5 400 "Avaliação obrigatória"
6 pede para preencher a avaliação

[avaliação completa]
7 pedido.aprovar()
8 atualizar estado = APROVADO
9 200 OK
10 mostra botão "Criar User Story"

Geração da User Story

11 clica "Criar User Story"
12 POST /pedidos/{id}/user-stories
13 pedido.gerarUserStory()

loop [até 2 tentativas, enquanto não houver resposta válida]
14 gerarEsboco(titulo, problema, anexos)
15 "Como..., eu preciso de..., para que..." + critérios Dado/Quando/Então
16 validar formato

alt [esboço válido]
17 usar esboço da IA

[IA falhou nas 2 tentativas]
18 usar descrição original do pedido (sem critérios)

19 guardar UserStory (estado = A_FAZER, ordemBacklog = último lugar do backlog) ligada ao Pedido de origem e aos anexos
20 id da User Story
21 201 Created
22 mostra o esboço para rever

Revisão

alt [edita e confirma]
23 edita texto e critérios
24 PUT /user-stories/{id}
25 guardar alterações
26 200 OK
27 story disponível no Product Backlog

[descarta o esboço]
28 clica "Descartar"
29 DELETE /user-stories/{id}
30 remover User Story
31 204 No Content

Gestor de Produto
Aplicação Web (React)
API Back-end (Django)
Base de Dados (PostgreSQL)
Serviço de IA

Código PlantUML

Diagrama2_Gerar_User_Stories.puml

```
@startuml Diagrama2_Gerar_User_Stories
skinparam shadowing false
skinparam defaultFontName Helvetica
skinparam defaultFontSize 13
skinparam dpi 96
skinparam sequenceMessageAlign left
skinparam responseMessageBelowArrow true
skinparam sequence {
  ParticipantBackgroundColor #E8F0FE
  ParticipantBorderColor #333333
  ActorBackgroundColor #E8F0FE
  ActorBorderColor #333333
  LifeLineBorderColor #888888
  GroupBackgroundColor #FFF4E0
}
autonumber
title Diagrama de Sequência 2 -- Aprovar pedido e gerar User Story (US 1.6, US 1.3)

actor "Gestor de Produto" as GP
participant "Aplicação Web\n(React)" as SPA
participant "API Back-end\n(Django)" as API
database "Base de Dados\n(PostgreSQL)" as DB
participant "Serviço de IA" as IA

== Aprovação ==
GP -> SPA : define esforço e impacto\ne clica "Aprovar"
SPA -> API : POST /pedidos/{id}/aprovar
activate API
API -> DB : obter Pedido
DB --> API : pedido
alt esforço ou impacto em falta
  API --> SPA : 400 "Avaliação obrigatória"
  SPA --> GP : pede para preencher a avaliação
else avaliação completa
  API -> API : pedido.aprovar()
  API -> DB : atualizar estado = APROVADO
  API --> SPA : 200 OK
  SPA --> GP : mostra botão "Criar User Story"
end
deactivate API

== Geração da User Story ==
GP -> SPA : clica "Criar User Story"
SPA -> API : POST /pedidos/{id}/user-stories
activate API
API -> API : pedido.gerarUserStory()
loop até 2 tentativas, enquanto não houver resposta válida
  API -> IA : gerarEsboco(titulo, problema, anexos)
  activate IA
  IA --> API : "Como..., eu preciso de..., para que..."\n+ critérios Dado/Quando/Então
  deactivate IA
  API -> API : validar formato
end
alt esboço válido
  API -> API : usar esboço da IA
else IA falhou nas 2 tentativas
  API -> API : usar descrição original do pedido\n(sem critérios)
end
```

```
API -> DB : guardar UserStory (estado = A_FAZER,\nordemBacklog = último lugar do backlog)\
    nligada ao Pedido de origem e aos anexos
DB --> API : id da User Story
API --> SPA : 201 Created
deactivate API
SPA --> GP : mostra o esboço para rever

== Revisão ==
alt edita e confirma
  GP -> SPA : edita texto e critérios
  SPA -> API : PUT /user-stories/{id}
  API -> DB : guardar alterações
  API --> SPA : 200 OK
  SPA --> GP : story disponível no Product Backlog
else descarta o esboço
  GP -> SPA : clica "Descartar"
  SPA -> API : DELETE /user-stories/{id}
  API -> DB : remover User Story
  API --> SPA : 204 No Content
end
@enduml
```